D20A / 37

Experiment No. 6

- **Aim**: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.
- **Course Outcomes (CO): CO3, CO5, CO6**

- **Theory**:

1. Relevance of `require` Statements in Solidity Programs

In Solidity, the `require` statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside `require` is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a Certificate Verification Smart Contract, `require` can be used to check:

- Whether the person calling the function is the authorized owner before registering a certificate (`require(msg.sender == owner, "Only owner can register");`).
- Whether a certificate ID has already been registered before allowing a new certificate to be added.
- Whether the certificate registration request satisfies the conditions defined in the smart contract.

Thus, `require` statements enforce security, correctness, and reliability in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: `mapping`, `storage`, and `memory`

Mapping:

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is `mapping(keyType => valueType)`.

For example:

```
mapping(uint256 => Certificate) public certificates;
```

Here, each certificate ID is mapped to a Certificate structure containing the student's name, course name, issue date, issuer address, and existence status.

Mappings are very useful for contracts like Certificate Verification, where certificate details need to be associated with unique certificate IDs. They provide efficient lookups, but unlike arrays, mappings do not have a length property and cannot be iterated over directly.

Storage:

In Solidity, storage refers to the permanent storage of the contract on the blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means that once written, it remains available unless explicitly modified. However, writing to blockchain storage consumes gas and is more expensive.

For example, the certificate information saved in the `certificates` mapping remains available throughout the contract's lifecycle.

Memory:

In contrast, memory is temporary storage used only during the execution of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that do not need to be permanently stored on the blockchain. It is generally cheaper than storage.

For example, the student name, course name, and issue date passed as string arguments to the `registerCertificate` function use the `memory` keyword.

Thus, a smart contract developer must balance between storage and memory to ensure efficiency and cost-effectiveness.

3. Use of `string` and `uint256` in Certificate Verification

In Solidity, different data types are used to store different kinds of information in a smart contract.

- **`string`:** A string is used to store text of variable length. In the Certificate Verification Smart Contract, it is used to store the student's name, course name, and issue date. It makes certificate details readable and easy to understand.
- **`uint256`:** This is an unsigned integer data type used to store non-negative whole numbers. In this experiment, it is used as the certificate ID to uniquely identify each certificate.
- **`address`:** This data type stores an Ethereum-compatible blockchain account address. In the smart contract, it is used to identify the contract owner and the account that issued the certificate.
- **`bool`:** This data type stores either `true` or `false`. It is used to indicate whether a

certificate has been registered.

The use of these data types helps organize certificate information and makes it easier to register, retrieve, and verify certificates.

In summary, `string` is preferred for readable certificate details, `uint256` is used for numeric certificate IDs, `address` identifies blockchain accounts, and `bool` indicates certificate validity.

- **Code**:

```
 // SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract CertificateVerification {

    address public owner;

    struct Certificate {
        string studentName;
        string courseName;
        string issueDate;
        address issuer;
        bool exists;
    }

    mapping(uint256 => Certificate) public certificates;

    constructor() {
        owner = msg.sender;
    }

    function registerCertificate(
        uint256 certificateId,
        string memory studentName,
        string memory courseName,
        string memory issueDate
    ) public {
        require(msg.sender == owner, "Only owner can register");
        require(!certificates[certificateId].exists,
            "Certificate already exists");

        certificates[certificateId] = Certificate({
            studentName: studentName,
            courseName: courseName,
            issueDate: issueDate,
            issuer: msg.sender,
            exists: true
        });
    }
```

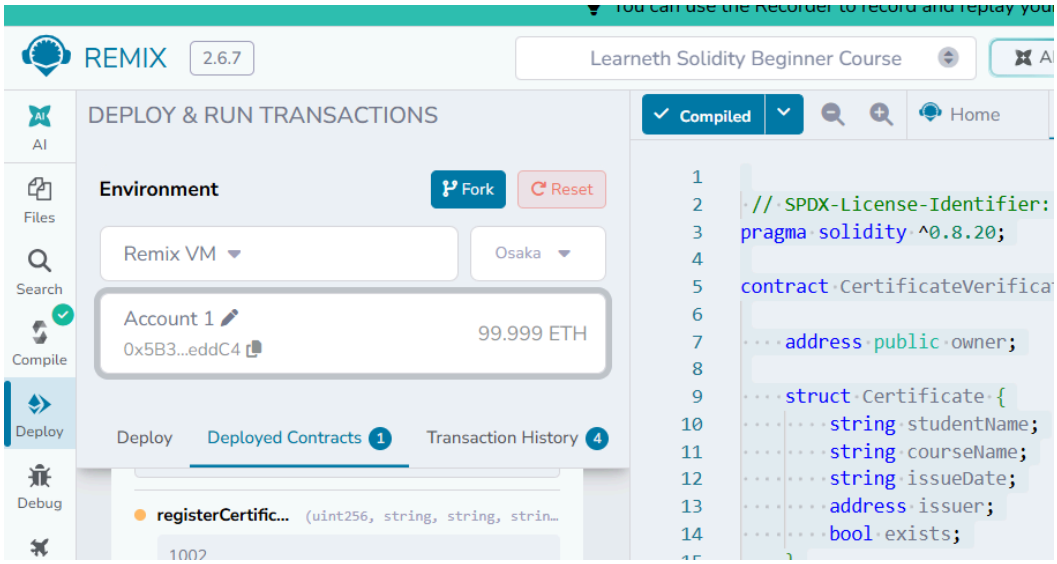
```
    function verifyCertificate(
        uint256 certificateId
    ) public view returns (bool) {
        return certificates[certificateId].exists;
    }

    function getCertificate(
        uint256 certificateId
    ) public view returns (
        string memory studentName,
        string memory courseName,
        string memory issueDate,
        address issuer,
        bool exists
    ) {
        Certificate memory cert = certificates[certificateId];

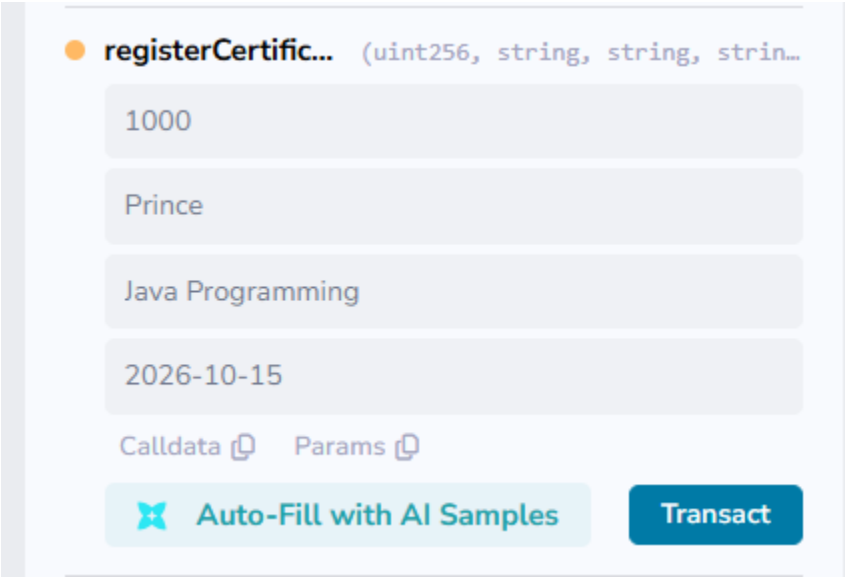
        return (
            cert.studentName,
            cert.courseName,
            cert.issueDate,
            cert.issuer,
            cert.exists
        );
    }
}
```

Output:

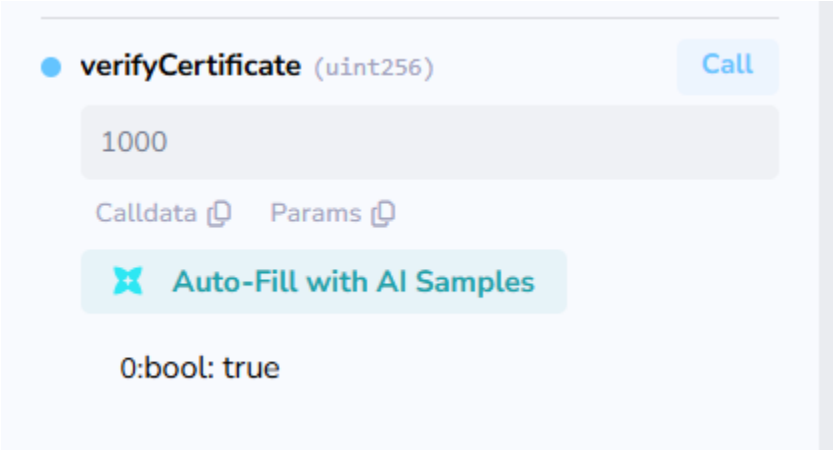
1. Compilation of the Certificate Verification Smart Contract



2. Registering a New Certificate on the Blockchain



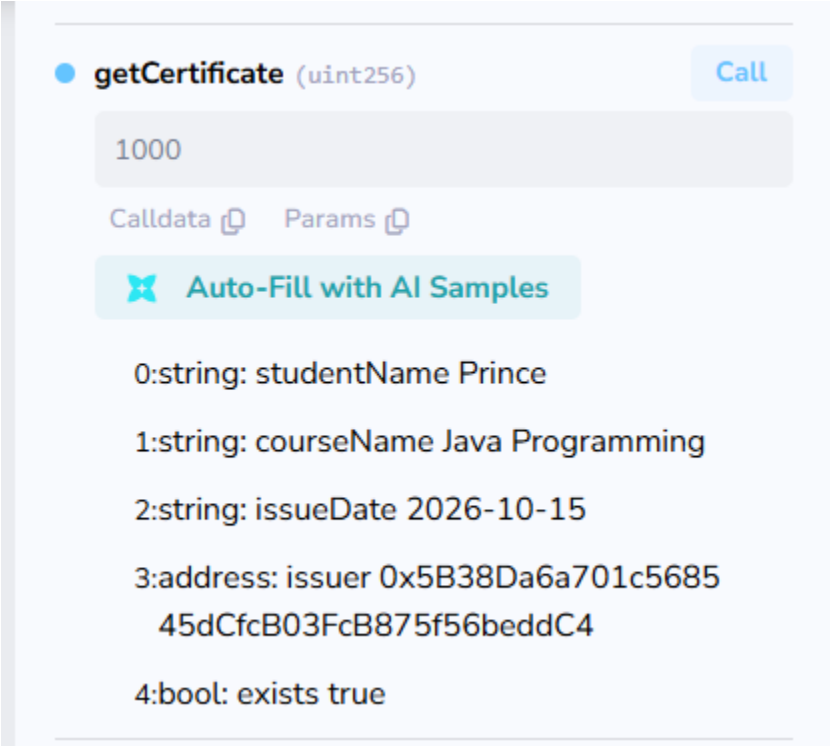
3. Verification of a Registered Certificate



4. Verification of unregistered certificate



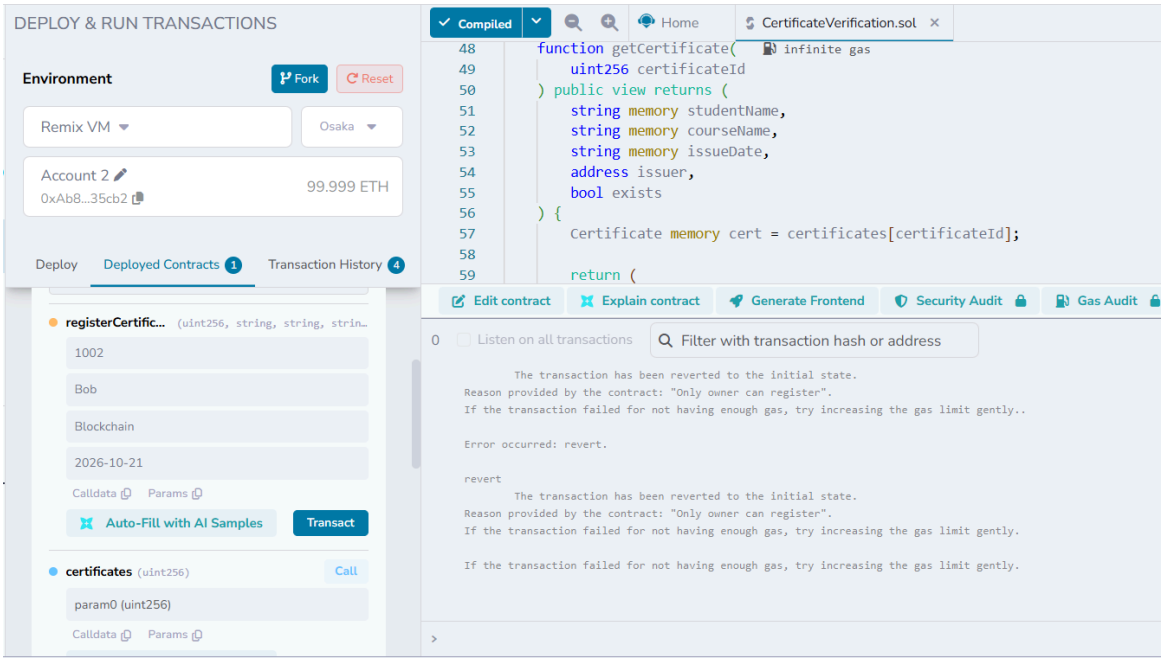
5. Retrieving the Stored Certificate Details



6. Testing Duplicate Certificate Registration



7. Testing unauthorized certificate registration



Conclusion

The experiment demonstrates how Solidity smart contracts can maintain certificate records and support verification through blockchain transactions. The use of mappings, structures, and require statements helps organize the data, prevent duplicate entries, and restrict registration to the authorized owner.